

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

Course Code:	CSA12	Course Title:	Computer Architecture for Multicore Systems
CO Assessed:	CO5	Assessment:	ASSESSMENT TOOL2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)
Weightage:	30%	Total Marks:	25
CO5 Statement:	<i>Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.</i>		
Student Name:	Kaviyaasri M	Reg.No.:	192312614
Department:	ECE	Date:	

ANNEXURE A
Pseudocode Writing Task (Algorithm Design)

1. Write pseudocode to design a DMA-based data transfer algorithm that moves data from an I/O device to memory while handling interrupts and minimizing CPU involvement.
2. Develop pseudocode for an interrupt handling system that prioritizes vectored interrupts from multiple devices and dispatches appropriate service routines.
3. Write pseudocode to select between memory-mapped I/O and I/O-mapped I/O dynamically based on latency and bandwidth requirements.
4. Design pseudocode for a bus arbitration algorithm that manages multiple devices communicating over PCIe, USB, and Thunderbolt interfaces.
5. Write pseudocode to implement a hybrid I/O communication mechanism that uses polling for low-speed devices and DMA with interrupts for high-speed devices.

Code of Conduct:

*I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

MARKS ALLOCATION

	Problem Understanding (20%)	Algorithm Design (30%)	Use of Control Structures (20%)	Pseudocode Structure (10%)	Completeness (20%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

1. DMA-Based Data Transfer Algorithm

Aim:

To design a DMA-based data transfer algorithm that transfers data from an I/O device to memory while handling interrupts and minimizing CPU involvement.

Pseudocode:

START

Initialize I/O device

Initialize DMA controller

Set source = I/O device

Set destination = memory

Set transfer_size

Configure DMA controller

Enable DMA transfer

Enable DMA completion interrupt

START DMA TRANSFER

WHILE transfer is not complete

 CPU performs other tasks

END WHILE

WHEN DMA interrupt occurs

 Save CPU status

 Check transfer status

 IF transfer is successful THEN

 Set transfer_complete = TRUE

 ELSE

 Handle transfer error

 END IF

 Clear DMA interrupt

 Restore CPU status

END INTERRUPT

Display "Data transfer completed"

STOP

Result:

The DMA controller transfers data directly from the I/O device to memory, reducing CPU involvement and improving data transfer efficiency.

2. Vectored Interrupt Handling System

Aim:

To develop an interrupt handling system that prioritizes vectored interrupts from multiple devices and dispatches the appropriate service routines.

Pseudocode:

START

Initialize interrupt controller
Assign priority to each device
Create interrupt vector table

WHILE system is running

 IF interrupt is received THEN

 Identify interrupting device
 Read interrupt vector
 Determine device priority

 IF multiple interrupts are pending THEN
 Select highest-priority interrupt
 END IF

 Save CPU context

 IF interrupt is hardware interrupt THEN
 Jump to corresponding service routine
 ELSE
 Handle software interrupt
 END IF

 Execute service routine
 Clear interrupt flag
 Restore CPU context

 END IF

END WHILE

STOP

Result:

The system identifies and services the highest-priority interrupt using the interrupt vector table and executes the corresponding service routine.

3. Dynamic Selection of Memory-Mapped I/O and I/O-Mapped I/O

Aim:

To dynamically select between memory-mapped I/O and I/O-mapped I/O based on latency and bandwidth requirements.

Pseudocode:

START

Input latency_requirement

Input bandwidth_requirement

IF latency_requirement is HIGH AND
bandwidth_requirement is HIGH THEN

 Select MEMORY-MAPPED I/O

 Use normal memory instructions

ELSE IF latency_requirement is LOW AND
bandwidth_requirement is LOW THEN

 Select I/O-MAPPED I/O

 Use special I/O instructions

ELSE

 Compare MMIO and I/O-Mapped I/O

 Select the method with better performance

END IF

Display selected I/O method

STOP

Result:

The system dynamically selects the appropriate I/O technique according to the required latency and bandwidth.

4. Bus Arbitration Algorithm for PCIe, USB and Thunderbolt

Aim:

To design a bus arbitration algorithm that manages multiple devices communicating through PCIe, USB, and Thunderbolt interfaces.

Pseudocode:

START

Initialize PCIe, USB and Thunderbolt interfaces

Create device request queue

WHILE system is running

 Check all device requests

 IF PCIe device requests bus THEN

 Add PCIe request to queue

 END IF

 IF USB device requests bus THEN

 Add USB request to queue

 END IF

 IF Thunderbolt device requests bus THEN

 Add Thunderbolt request to queue

 END IF

 Apply priority and round-robin arbitration

 Select next device from queue

 IF device has HIGH priority THEN

 Grant bus access immediately

 ELSE

 Grant bus access according to round-robin order

 END IF

 Transfer data

 Release bus

 Update queue

END WHILE

STOP

Result:

The bus arbitration algorithm provides efficient and fair access to PCIe, USB, and Thunderbolt devices while preventing device starvation.

5. Hybrid I/O Communication Mechanism

Aim:

To implement a hybrid I/O communication mechanism that uses polling for low-speed devices and DMA with interrupts for high-speed devices.

Pseudocode:

START

Initialize all I/O devices

FOR each device

 Determine device speed

 IF device is LOW-SPEED THEN

 Enable polling

 WHILE device is active

 Check device status

 IF data is available THEN

 Read data

 Process data

 END IF

 END WHILE

 ELSE IF device is HIGH-SPEED THEN

 Configure DMA controller

 Set source and destination

 Enable DMA

 Enable DMA completion interrupt

 START DMA TRANSFER

 WHEN DMA interrupt occurs

 Check transfer status

 IF transfer is successful THEN

 Process transferred data

 ELSE

 Handle transfer error

 END IF

 Clear interrupt

 END WHEN

 END IF

END FOR

STOP

Result:

The hybrid mechanism uses polling for low-speed devices and DMA with interrupts for high-speed devices, reducing CPU overhead and improving overall I/O performance.